HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 10

Code-Based Agent Development (Copilot SDK) & Simple RAG Flow

Build a working agent in code, then make it trustworthy: wire tools, understand the execution loop and its guardrails, and ground the agent in your own documents with a simple retrieval step.

DURATION
2 hours

FORMAT
Concepts · build lab

PRIMARY TOOL
Copilot SDK + sample doc set

How We'll Spend These Two Hours

One agent, built up in layers: tools first, then the loop, then your own knowledge.



01

Chat to Agent & the Copilot SDK

15 min

What changes when a model gets tools and a loop, and where the SDK fits.



02

Anatomy of a Single Agent

25 min

Instructions, model, session and tools — and how to wire a tool well.



03

The Loop, Guardrails & Use Cases

15 min

Plan, act, observe; the control points; and real-world single-agent jobs.



04

Grounding the Agent with RAG

25 min

Index, retrieve, ground — and the failure points to watch.



05

Lab: Agent, then RAG

35 min

Build the agent, run a baseline, add retrieval, and compare grounded with ungrounded.



06

Best Practices & Wrap-up

5 min

Do and avoid, then the handoff to observability and ROI in Module 11.

Module 10 in the 12-Module Journey

Modules 07 and 09 used agents someone else built; here you build the agent yourself.

FOUNDATION

1 Objectives & GenAI Recap

2 Copilot Advanced

3 Prompt & Context Eng.

BUILD & VALIDATE

4 Spec-Driven Dev (Spec Kit)

5 SDD to Full SDLC

6 Test Strategy & Validation

EXTEND WITH TOOLS & AGENTS

7 MCP & Custom MCP Server

8 PR Automation & LLM-Judge

9 No-Code Agents (Rovo)

10 Code Agents & Simple RAG

GOVERN

11 Observability, ROI & Gov.

CAPSTONE

12 Capstone & Roadmap

M03

Instructions and retrieved context are context engineering

M07

Agent tools can be MCP servers you already built

M09

The studio agent idea, now authored in code

M11

Agent Prism traces what this agent does and costs

Why this matters: RAG is taught here as a source of context for the agent you just built, not as a separate architecture.

From Chat to Agent to Grounded Agent

Each step adds one capability — and one new thing that can go wrong.

Chat

Answers from what the model already knows

Risk: Stale, generic, no access

Agent with tools

Calls functions to look things up or act

Risk: Wrong tool, unsafe action

Agent loop

Plans, acts, observes, repeats until done

Risk: Runaway steps and cost

Grounded agent (RAG)

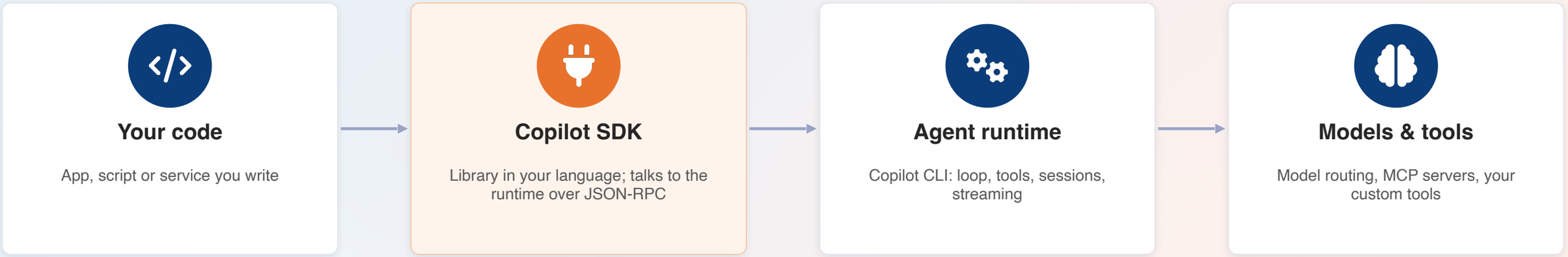
Retrieves your documents before answering

Risk: Missed or misread sources

Today's path: we climb the ladder in order, building one agent and adding a capability at each step. Skipping a rung means debugging two problems at once.

Where the Copilot SDK Fits

Your code gets programmatic access to the same agent runtime behind Copilot CLI.



WHAT YOU NEED

- **Copilot CLI installed and signed in**
- **Copilot subscription — or your own model key (BYOK)**
- **A supported language: Node.js, Python, Go or .NET**

Check before you design: the SDK is new and moving. Confirm current status, languages, API names and billing in the official repository and GitHub docs before building on it.

Anatomy of a Single Agent

Five parts. Most agent bugs trace back to one of them being vague.

Instructions

Role, scope, output format, what to refuse

Model

The reasoning engine you choose or are given



Session & memory

Conversation state across turns

Tools

Functions the agent may call

Plus: hooks and permissions that approve, deny or log tool calls

Debug in this order: instructions first, tools second, model last. Changing the model rarely fixes an unclear job.

Wiring a Tool: Name, Description, Inputs, Handler

The model only sees the description and the input shape. Write them for the model.

PSEUDOCODE — MAP TO YOUR SDK'S REAL API NAMES

tool + agent (pseudocode, illustrative)

```
tool get_build_status:
  description: "Status and failing step for a build.
                Read-only. id looks like b-1234."
  inputs:      build_id: string (pattern b-####)
  handler:     return ci_api.status(build_id)

agent = create_agent(
  instructions = "Explain build failures. Use tools;
                  never guess. Say if unsure.",
  tools = [get_build_status])

agent.send("Why did build b-1234 fail?")
```

One tool, one job

Narrow, clearly named

Describe for the model

Say when to use it and what it returns

Typed, validated inputs

Reject bad arguments in the handler

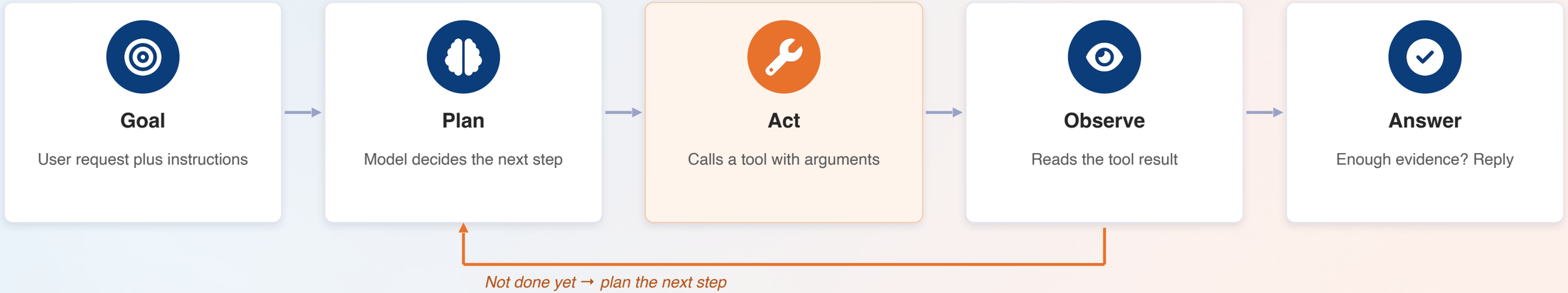
Small, structured output

Return what is needed, not a dump

Reuse: the tool-design rules from Module 07 apply unchanged. A tool is a tool whether it is an MCP server or a function in your code.

The Execution Loop: Plan, Act, Observe, Repeat

The agent keeps going until it can answer — or a limit stops it.



STOP CONDITIONS TO SET

- **Maximum steps**
- **Time budget**
- **Tool allow-list**
- **Approval needed**

Why limits matter: every loop turn calls a model, and tool-heavy agents use far more of your budget than a single prompt. Set limits before you run it unattended.

Where to Put the Controls Around the Loop

The SDK lets you intercept tool use, so enforcement lives in code, not in hope.

Before a tool call

Permission hook: approve, deny or ask a human

Block writes and unknown tools

Inside the tool

Validate inputs; least-privilege credentials

Reject malformed or out-of-range args

After a tool call

Log the call and result; filter sensitive data

Trace for later review

Around the session

Step, time and cost limits; isolated workspace

Stop runaway behaviour

Same principle as Modules 07–09: least privilege, a human at the risky steps, and a record of what happened. Hook names and options vary — read the SDK docs for what is available.

Real-World Single Agents: Small Jobs with Real Tools

Start with one narrow job, two or three tools and a human who checks the result.

Build-failure explainer

TOOLS (ILLUSTRATIVE)

`get_build_status` · `get_log_excerpt`

HUMAN CHECK

Developer reads the cause and fix

Release-notes drafter

TOOLS (ILLUSTRATIVE)

`list_merged_prs` · `get_pr_summary`

HUMAN CHECK

Release owner edits before publishing

Ticket summariser

TOOLS (ILLUSTRATIVE)

`get_ticket` · `list_comments`

HUMAN CHECK

Support lead confirms the summary

Runbook helper

TOOLS (ILLUSTRATIVE)

`search_runbook (needs your docs)`

HUMAN CHECK

On-call engineer verifies each step

Notice the fourth one: a runbook helper is only as good as the runbooks it can read. That is the gap retrieval fills — and where we go next.

Why Ground the Agent: Your Knowledge Is Not in the Model

Private, recent or niche documents are invisible to a model unless you bring them in.

WITHOUT RETRIEVAL

Q: What is our rollback window for the payments service?

A: “Typically 30 minutes.” (confident, generic, unsourced)

Plausible-sounding, possibly wrong, nothing to check.

WITH RETRIEVAL (illustrative)

Q: What is our rollback window for the payments service?

A: “Per runbook-payments.md §4: <value from the document>.” — or “I couldn't find this in the documents.”

Sourced, checkable, and honest when the answer is missing.

REAL-WORLD GROUNDED AGENTS

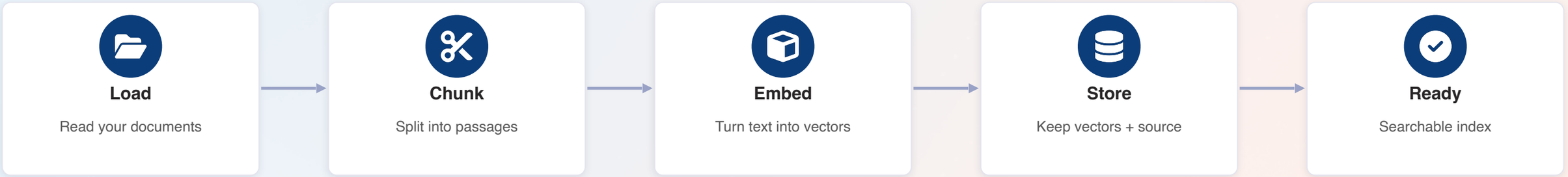
- **Policy and process Q&A**
- **Runbook assistant for on-call**
- **Internal API docs helper**

Definition: retrieval-augmented generation (RAG) fetches relevant passages from your documents and gives them to the model as context before it answers.

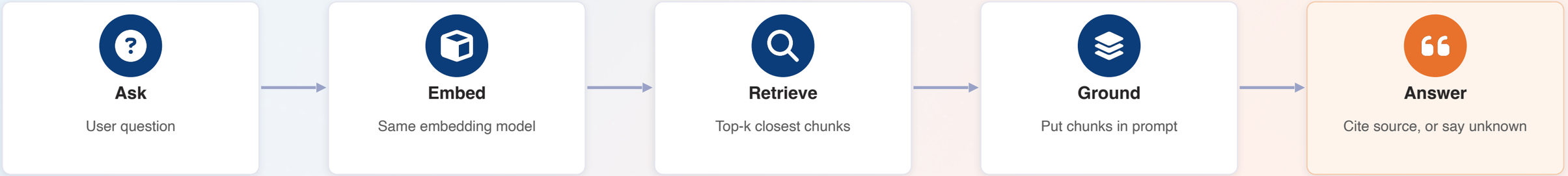
The Simple RAG Pipeline: Index Once, Retrieve Every Time

Two lanes — one prepares your documents, the other answers each question.

INDEX (ONCE, AND WHENEVER DOCUMENTS CHANGE)



QUERY (EVERY QUESTION)



Keep it simple first: a small document set, one embedding model, top-k retrieval and source names in the answer. Add re-ranking or hybrid search only when you can show the simple version fails.

Plugging Retrieval into the Agent: Retrieval Is Just Another Tool

The agent decides when to search; your instructions decide what it does with the result.

Agent

search_docs (pseudocode)

```
tool search_docs(query: string, k: int = 4)
  return [{ text, source, chunk_id } ...]
```

- **Chunk size —** Big enough for meaning, small enough to match
- **Top-k —** Enough to cover the answer, not so many that noise wins
- **Metadata —** Keep file name and chunk number with every passage
- **Instructions —** “Answer only from results; cite sources; say if not found”

TWO WAYS TO CALL IT

Agent decides (tool): Flexible; may skip retrieval when it should search

Always retrieve first: Predictable; wasteful for questions that need no documents

Tip: start with “agent decides” for a tool-using agent, and log whether it searched. A missed search is a bug you can see in the trace.

Where RAG Goes Wrong — and the First Thing to Try

Research on real RAG systems names recurring failure points. Check them in this order.

FAILURE	SYMPTOM	FIRST FIX TO TRY
Answer not in the documents	Agent answers anyway instead of declining	Instruct: say “not found”; test with unanswerable questions
Right passage ranked too low	Answer exists but is not retrieved	Adjust chunking, top-k; add metadata
Retrieved but not used	Passage dropped or buried in the context	Reduce noise; fewer, better chunks
In context, not extracted	Model misses or mixes up the facts	Clearer instructions; cleaner source text
Wrong format or specificity	Too vague, too detailed or wrong shape	State the output format and level of detail

Habit: build a small question set — including questions the documents cannot answer — and re-run it after every change. RAG quality is measured, not assumed.

Lab: Build the Agent, Then Ground It

LAB 1 · 35 min

Build a minimal agent, measure it ungrounded, add retrieval, and compare.

Goal Build a code-based agent with one tool, add a simple retrieval step over a small document set, and compare grounded with ungrounded answers.

STEPS

1. From the starter template, create an agent with clear instructions and one read-only tool. Run it on one question and read the trace.
2. Baseline: ask five questions about the sample documents, including two they cannot answer. Save the answers without retrieval.
3. Index the documents (load, chunk, embed, store). Add a search_docs tool that returns passages with source names, and update the instructions to cite or say “not found”.
4. Re-run the same five questions. Compare with the baseline and label each failure with the point where it broke.

FILES YOU CREATE

```
repo/
lab10/
  agent/
  docs/
  baseline.md
  grounded.md
  comparison.md
```

DONE WHEN

comparison.md shows each question grounded vs ungrounded, whether sources were cited, and at least one failure traced to a pipeline stage.

Best Practices for Code-Based Agents and RAG

Build in layers, constrain early, and measure with questions you wrote down.

DO THIS	AVOID THIS
Give the agent one narrow job and few tools	Hand the agent broad tools or write access on day one
Write tool descriptions and typed inputs for the model	Change the model before fixing unclear instructions
Set step, time and permission limits before running	Let the loop run with no step or cost limit
Log every tool call so behaviour can be reviewed	Dump whole documents into the prompt
Cite sources and say “not found” when retrieval is empty	Judge RAG quality from one good-looking answer
Keep a fixed question set and re-run it after changes	Add re-ranking and hybrid search before the simple version fails

Golden rule: an agent is code that spends money and takes actions — treat it with the same review, limits and logging as any other production code.

What You Can Do Now

Six outcomes — from a first tool-using agent to one grounded in your own documents.

- ✓ Explain how chat, tools, the loop and retrieval build on each other
- ✓ Describe where the Copilot SDK sits and what it needs to run
- ✓ Wire a well-described, validated tool into a single agent
- ✓ Place limits and approval hooks around the execution loop
- ✓ Build a simple index and retrieval step as an agent tool
- ✓ Compare grounded with ungrounded answers and trace failures

NEXT: MODULE 11 — OBSERVABILITY, ROI & GOVERNANCE (AGENT PRISM) · 1.5 HOURS

Trace what your agents do, find token and cost leakage, and connect AI-assisted work to quality, cost and ROI.

TODAY'S BUILD-UP



Questions & Discussion

Module 10 — Code-Based Agent Development (Copilot SDK) & Simple RAG Flow